Software de Aerolineas Low-Cost

Jeremias Benitez Sanitillo

Escuela Técnica N°32 DE 14 “Gral. José de San Martin”, CABA

Proyecto Interdisciplinario

Nahuel Acosta y

Gabriel Abello Tuinyla

CABA, Argentina

6 de Agosto de 2026

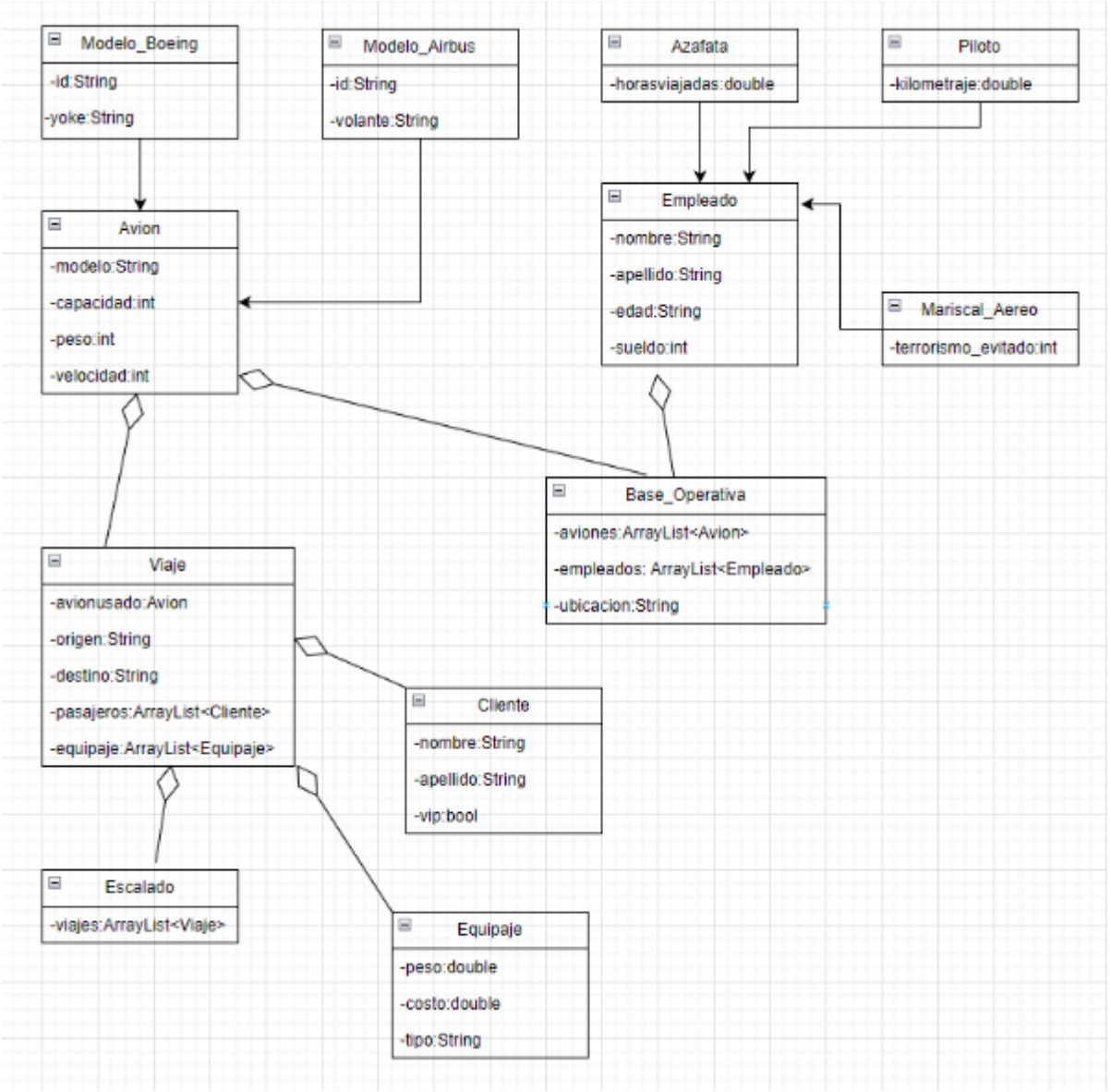
La presente carpeta de campo tiene como objetivo registrar de manera detallada las actividades realizadas por Jeremias Benitez Santillo en el marco de las materias: Programación Orientada a Objetos, Redes, Análisis de Sistemas, Base de Datos y Proyecto Informático II. El propósito principal es llevar un control organizado de las tareas, tanto individuales como grupales, a fin de documentar el proceso de trabajo, los avances y las dificultades encontradas.

Los registros incluirán la siguiente información básica en cada entrada: fecha, lugar donde fue realizada la actividad, hora en la que se realizó la actividad, así como una descripción objetiva de lo que se hizo. Además, se añadirán observaciones o comentarios que permitan reflexionar sobre el desarrollo del trabajo.

Este documento servirá como evidencia del compromiso y participación de Jeremias Benitez Santillo, y permitirá evaluar de forma clara el progreso alcanzado durante el período establecido.

Fecha: 30/06/2026

Creación del primer boceto del UML



lo terminamos cambiando con el avance del proyecto

Fecha: 3/07/2026

estuve investigando más a detalle como es una compañía de transporte aéreo y revisando las actividades que solicitaron los profesores para la primera entrega

Fecha 10/07/2026

revisamos la entrevista entregada por chodos y empezamos a ver en detalle todos los diagramas

Fecha 11/07/2026

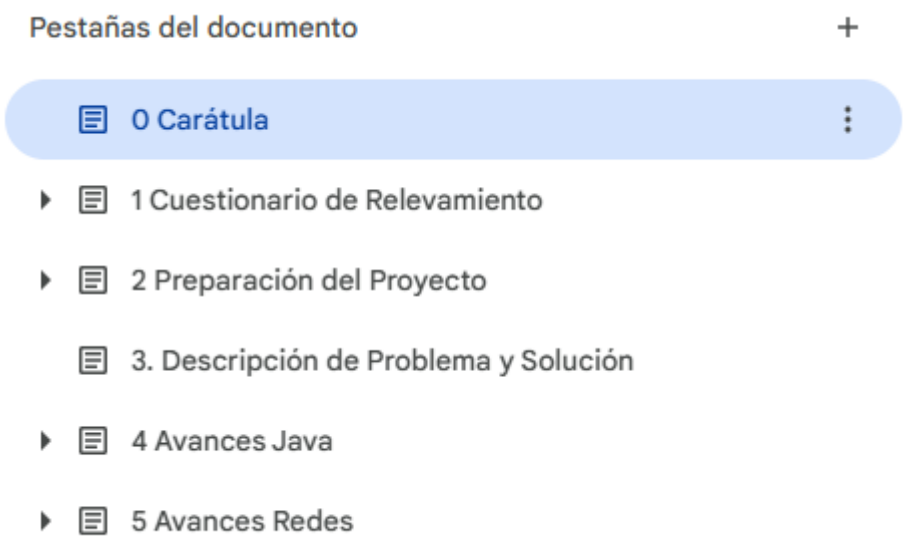
Se creó el borrador de la carpeta de campo grupal con toda la información recopilada de cada uno hasta el momento que durante el largo del proyecto
Fecha 11/07/2026

Busque el presupuesto de varias computadoras y elegí entre las más económicas para hacer un presupuesto de las primeras computadoras para cada sector

Fecha 07/08/2026

Se arregló el borrador de la carpeta de campo con toda la información recopilada descartando los módulos que ya no son utilizados para seguir avanzando y solo dejando los más importantes para el mvp, subiendo toda la información a esa carpeta de campo.

Imagen adjunta de como están dispuestas las pestañas del borrador de la carpeta de campo grupal:



Fecha: 10/08/2026

Se trabajó sobre el registro de campo del proyecto, específicamente en la normalización y depuración del transcrito de la entrevista realizada por el compañero Joshua chodos al piloto de Flybondi octavio. El objetivo de la clase fue corregir los errores de formato del documento original y adaptar las respuestas técnicamente para mejor legibilidad , asegurando la fidelidad de los datos relevados para el módulo de Piloto.

A continuación, se adjunta el segmento corregido y validado de la entrevista:

Entrevista de Relevamiento
Entrevistado: Octavio (Piloto Comercial)

¿Cuál es la duración del viaje y el trayecto más largo que realizan habitualmente?
En la aviación comercial realizamos principalmente trayectos regionales o de cabotaje, lo que implica hacer múltiples vuelos cortos en una misma jornada operativa y evitar por completo los cruceros oceánicos; actualmente, mi tramo más largo es la ruta entre Londres y Larnaca, en Chipre, que toma unas 5 horas de vuelo entre ida y vuelta.

¿Cuántos pilotos y personal técnico se necesitan en cabina según el tipo de ruta establecida?
Hoy en día se necesitan exactamente dos personas en la cabina de mando, cumpliendo los roles de piloto y copiloto, ya que la figura del técnico de vuelo desapareció por la modernización tecnológica, aunque si la ruta es de larga distancia el sistema exige sumar más pilotos para los turnos de relevo y descanso.

¿Cómo se maneja la cantidad y frecuencia de las escalas en la operación diaria?
La cantidad y frecuencia de las escalas se maneja puramente según la demanda de pasajeros de cada destino, operando múltiples frecuencias diarias en ciudades grandes como Buenos Aires o Córdoba, mientras que en lugares con menos movimiento como Río Grande o destinos turísticos como Bariloche la frecuencia se adapta según la temporada.

¿Qué impacto operativo tienen los diferentes precios y tasas de cada aeropuerto en la ruta?
Sobre el impacto operativo que tienen los diferentes precios y tasas de cada aeropuerto en la ruta no te sabría decir, ya que no poseo información sobre ese tema en este momento.

¿Cuáles son los principales costos directos asociados exclusivamente a la operación en el aire?
Los principales costos directos de la operación aérea cuando el avión ya está en el aire son el combustible especial que utilizamos, conocido como Jet A-1 (kerosén de aviación), y los gastos

Fecha 11/08/2026

Al tener el diagrama de Gantt nos facilitó nuestra forma de trabajar. Después de una larga jornada junto a Joaquín estructurando el proyecto y agregando en nuestras tablas de seguimiento y asignación de responsabilidades, nos pudimos ingeniar más con los tiempos para la finalización de actividades.

Utilizando las herramientas de Google Docs y Google Sheets de forma colaborativa, pudimos hacer que esas tablas de seguimiento se puedan entender claramente. Durante la sesión,

realizamos la estimación de tiempos de ejecución para cada bloque técnico y procedimos con la asignación formal de responsables y encargados para cada actividad, dejando consolidada la estructura analítica antes de la generación definitiva del diagrama de Gantt.

Tarea	Símbolo	Dependencias	Participante	Duración (Días)	Estado	3
Recoleccion de informacion	A0	NINGUNO	Joaquin Ferramola, Juan Molinas, Jeremias Benitez, Joshua Chodoz, Gonzalo Delgado	5	Completada	
Entrevista	A1	A0	Joshua Chodoz	1	Completada	
Pasar la entrevista a limpio	A2	A1	Juan Molinas	1	Completada	
Relevamiento	A3	A1	Joaquin Ferramola, Juan Molinas, Jeremias Benitez, Joshua Chodoz, Gonzalo Delgado	1	Completada	
Análisis foda	A5	A3	Gonzalo Delgado	1	Completada	
Modelo gantt	A4	A3	Joaquin Ferramola, Gonzalo Delgado, Juan Molinas	2	Completada	
Requisitos funcionales	A8	A3	Jeremias Benitez	2	No iniciada	
Modelo pert	A6	A4	Joaquin Ferramola	1	No iniciada	
Modelo scrum	A7	A4	Juan Molinas	1	No iniciada	
Requisitos no funcionales	A9	A8	Joshua Chodoz	2	No iniciada	
Documentacion	A10	A8, A9	Juan Molinas	3	No iniciada	
Calcular el minimo producto viable	A11	A10	Gonzalo Delgado	1	Bloqueada	
DER	B0	A11		1	No iniciada	
Creacion de la base de datos	B1	B0		3	No iniciada	

En este primer modelo todavía no estaban bien definidas las actividades para cada uno pero íbamos en ese camino.

Fecha 18/08/2026

Durante esta última jornada enfocada en la documentación, establecimos y terminamos los detalles de la portada oficial para nuestra carpeta de campo grupal. Con el propósito de proyectar una imagen linda sobre nuestro grupo, agregamos de manera formal los datos de todos los integrantes del equipo, definimos con precisión el título oficial del software desarrollado, especificamos los diferentes módulos que lo componen. De este modo, logramos terminar y presentar un documento prolijo y perfectamente presentable para las revisiones y entregas del proyecto.

Proyecto Interareal

Temática: Aerolineas Low-Cost (LCC)

Integrantes:
- Juan Molinas (L)
- Joaquín Ferramola
- Gonzalo Delgado
- Jeremías Benitez
- Joshua Chodoz

Índice:

1 Cuestionario de Relevamiento
  1.1 Entrevista
  1.2 Módulo: Piloto
  1.3 Módulo: Gestión de Vuelos
  1.4 Módulo: Encargado de Ganancias (Revenue Manager)
  1.5 Módulo: Control Operativo
  1.6 Módulo: Analista de Costos
  1.7 Módulo: Equipaje
2 MVP (Minimum Viable Product)
3 Que soluciona nuestro programa
4 Java
  4.1 UML
5 Redes
  5.1 Presupuesto
  5.2 Topografía

Fecha: 19/08/2026

Concluimos la sesión de trabajo redactando y validando formalmente el listado definitivo de requerimientos. Logramos consolidar un total de 50 especificaciones clave, divididas equitativamente: 25 requisitos funcionales como el control de ventanas para el proceso de check-in, la lógica de asignación aleatoria de asientos y el cálculo de los niveles de equipaje, y 25 requisitos no funcionales, orientados a asegurar un funcionamiento óptimo de la aplicación en Java como la alta tolerancia a la concurrencia, transacciones ACID para prevenir la duplicidad de asientos y un consumo de memoria RAM sumamente eficiente.

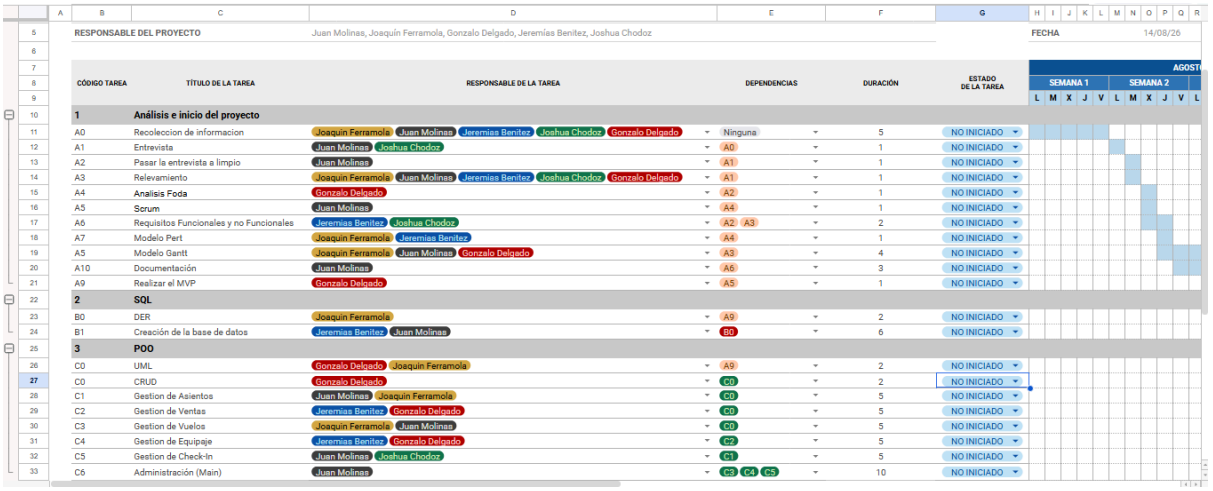
Asimismo, avanzamos en la reestructuración del documento preliminar de la carpeta de campo grupal. Integramos de forma ordenada toda la información técnica que ya ha sido

validada hasta el momento y suprimimos por completo las secciones asociadas a aquellos módulos que se descartaron previamente, evitando así cualquier redundancia y garantizando un avance limpio, coherente y estructurado para las próximas fases del proyecto.

Fecha 21/08/2026

Realizamos una reingeniería integral de la planificación temporal del proyecto. Tras evaluar la primera versión del diagrama de Gantt, tomamos la decisión de descartar dicho documento debido a las notables complejidades que implicaba su mantenimiento manual y su tosca flexibilidad para reflejar de manera ágil las modificaciones imprevistas en las tareas.

Como alternativa, procedimos a estructurar un nuevo cronograma más dinámico, empleando para ello una plantilla especializada de diagrama de Gantt en formato de hoja de cálculo. En esta plataforma integramos la totalidad de las actividades correspondientes a las fases de Análisis, Diseño, Desarrollo, Pruebas e Implementación que ya habían sido aprobadas para el MVP. Esto nos permitió aprovechar al máximo las funciones de automatización de la herramienta para calcular con precisión las barras de duración en días, monitorear los porcentajes de avance por cada bloque técnico y visualizar de forma clara la ruta crítica del desarrollo.



RESPONSABLE DEL PROYECTO		Juan Molinas, Joaquín Ferramola, Gonzalo Delgado, Jeremías Benitez, Joshua Chodoz				FECHA	14/08/26
CÓDIGO TAREA	TÍTULO DE LA TAREA	RESPONSABLE DE LA TAREA		DEPENDENCIAS	DURACIÓN	ESTADO DE LA TAREA	AGOSTO — SEMANA 1: L M X J V — SEMANA 2: L M X J V L
1	Análisis e inicio del proyecto						
A0	Recoleccion de informacion	Joaquin Ferramola, Juan Molinas, Jeremias Benitez, Joshua Chodoz, Gonzalo Delgado		Ninguna	5	NO INICIADO	
A1	Entrevista	Juan Molinas, Joshua Chodoz		A0	1	NO INICIADO	
A2	Pasar la entrevista a limpio	Juan Molinas		A1	1	NO INICIADO	
A3	Relevamiento	Joaquin Ferramola, Juan Molinas, Jeremias Benitez, Joshua Chodoz, Gonzalo Delgado		A1	1	NO INICIADO	
A4	Analisis Foda	Gonzalo Delgado		A2	1	NO INICIADO	
A5	Scrum	Juan Molinas		A4	1	NO INICIADO	
A6	Requisitos Funcionales y no Funcionales	Jeremias Benitez, Joshua Chodoz		A2, A3	2	NO INICIADO	
A7	Modelo Pert	Joaquin Ferramola, Jeremias Benitez		A4	1	NO INICIADO	
A5	Modelo Gantt	Joaquin Ferramola, Juan Molinas, Gonzalo Delgado		A3	4	NO INICIADO	
A10	Documentación	Juan Molinas		A6	3	NO INICIADO	
A9	Realizar el MVP	Gonzalo Delgado		A5	1	NO INICIADO	
2	SQL						
B0	DER	Joaquin Ferramola		A9	2	NO INICIADO	
B1	Creación de la base de datos	Jeremias Benitez, Juan Molinas		B0	6	NO INICIADO	
3	POO						
C0	UML	Gonzalo Delgado, Joaquin Ferramola		A9	2	NO INICIADO	
C0	CRUD	Gonzalo Delgado		C0	2	NO INICIADO	
C1	Gestion de Asientos	Juan Molinas, Joaquin Ferramola		C0	5	NO INICIADO	
C2	Gestion de Ventas	Jeremias Benitez, Gonzalo Delgado		C0	5	NO INICIADO	
C3	Gestion de Vuelos	Joaquin Ferramola, Juan Molinas		C0	5	NO INICIADO	
C4	Gestion de Equipaje	Jeremias Benitez, Gonzalo Delgado		C2	5	NO INICIADO	
C5	Gestion de Check-In	Juan Molinas, Joshua Chodoz		C1	5	NO INICIADO	
C6	Administración (Main)	Juan Molinas		C5, C4, C5	10	NO INICIADO	

Fecha 24/08/2026

Realizamos la creación y el diseño estructural de la base de datos del proyecto, tomando como punto de partida el modelo inicial que propuse originalmente para el equipo. A partir de esa base, me encargué de refinarla y modificarla de manera integral para optimizar su arquitectura, incorporar nuevos campos y establecer relaciones sólidas entre las tablas.

En su primer diseño, la estructura contemplaba un total de trece entidades fundamentales: Aeropuerto, Avion, Departamento, Pasajero, Servicio, Usuario, Asiento, Empleado, Vuelo, Boleto, Boleto_Servicio, Pago y CheckIn.

Tabla	Acción	Filas	Tipo	Cotejamiento	Tamaño	Residuo a depurar
aeropuerto	Examinar Estructura Buscar Insertar Vaciar Eliminar	8	InnoDB	utf8mb4_general_ci	32.0 KB	-
asiento	Examinar Estructura Buscar Insertar Vaciar Eliminar	720	InnoDB	utf8mb4_general_ci	112.0 KB	-
avion	Examinar Estructura Buscar Insertar Vaciar Eliminar	4	InnoDB	utf8mb4_general_ci	16.0 KB	-
boleto	Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_general_ci	64.0 KB	-
boleto_servicio	Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_general_ci	32.0 KB	-
checkin	Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_general_ci	80.0 KB	-
departamento	Examinar Estructura Buscar Insertar Vaciar Eliminar	3	InnoDB	utf8mb4_general_ci	32.0 KB	-
empleado	Examinar Estructura Buscar Insertar Vaciar Eliminar	6	InnoDB	utf8mb4_general_ci	80.0 KB	-
pago	Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_general_ci	48.0 KB	-
pasajero	Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_general_ci	64.0 KB	-
servicio	Examinar Estructura Buscar Insertar Vaciar Eliminar	4	InnoDB	utf8mb4_general_ci	32.0 KB	-
usuario	Examinar Estructura Buscar Insertar Vaciar Eliminar	7	InnoDB	utf8mb4_general_ci	48.0 KB	-
vuelo	Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_general_ci	64.0 KB	-
13 tablas	Número de filas	752	InnoDB	utf8mb4_general_ci	704.0 KB	0 B

La tabla Aeropuerto almacena los datos de los aeropuertos, incluyendo su nombre, país y ciudad.

	IDAeropuerto	nombre	pais	ciudad
Editar Copiar Borrar	1	Aeropuerto Internacional do Macaquinho	Brasil	São Paulo
Editar Copiar Borrar	2	Aeropuerto Internacional de Aura	Brasil	Río de Janeiro
Editar Copiar Borrar	3	Aeropuerto Internacional Charlie Kirk	México	Ciudad de México
Editar Copiar Borrar	4	Aeropuerto Internacional 67	México	Cancún
Editar Copiar Borrar	5	Aeropuerto Internacional Quiero Queque	Argentina	Buenos Aires
Editar Copiar Borrar	6	Aeropuerto Internacional Quiero Pico	Chile	Santiago
Editar Copiar Borrar	7	Aeropuerto Internacional Pablo Escobar	Colombia	Bogotá
Editar Copiar Borrar	8	Aeropuerto Internacional Faraón	Perú	Lima

La tabla Avion registra los aviones disponibles, sus modelos y capacidades. Por otro lado, Departamento organiza las áreas de trabajo de la aerolínea, mientras que Servicio almacena los servicios adicionales que pueden contratar los pasajeros, junto con sus precios y descripciones.

	IDAvion	modelo	capacidad
Editar Copiar Borrar	AV-A320-01	Airbus A320neo	180
Editar Copiar Borrar	AV-A320-02	Airbus A320neo	180
Editar Copiar Borrar	AV-B737-01	Boeing 737 MAX 8	180
Editar Copiar Borrar	AV-B737-02	Boeing 737 MAX 8	180

	IDDepartamento	nombre	tipo
Editar Copiar Borrar	1	Piloto	Aéreo
Editar Copiar Borrar	2	Seguridad	Terrestre
Editar Copiar Borrar	3	Check-in	Terrestre

	IDServicio	nombre_servicio	precio	descripcion
Editar Copiar Borrar	1	Equipaje Extra	16700.00	Permite llevar hasta 2 valijas extra, una en la bo...
Editar Copiar Borrar	2	Embarque Prioritario	10000.00	Permite entrar primero al avión.
Editar Copiar Borrar	3	Menú Premium	20000.00	Ofrece un menú con comidas y bebidas para consumir...
Editar Copiar Borrar	4	Selección de Asiento	15000.00	Permite elegir tu asiento y que no te sea seleccio...

La tabla Usuario administra las cuentas de acceso al sistema y distingue a los usuarios según sus privilegios. La tabla Pasajero almacena la información personal de los clientes que utilizan la aerolínea, mientras que Empleado registra los datos de los trabajadores y los vincula con sus respectivos departamentos.

	IDUsuario	nombre_usuario	email	contraseña_usuario	privilegios_empleado
Editar Copiar Borrar	1	Chorifly_Admin	admin@chorifly.com	Choriflyadmin1@!	1
Editar Copiar Borrar	2	la_cobra	lacobra@gmail.com	Cobra123!	1
Editar Copiar Borrar	3	davo_xeneize	davoxeneize@gmail.com	Davo123!	1
Editar Copiar Borrar	4	goncho_batan	gonchobatan@gmail.com	Goncho123!	1
Editar Copiar Borrar	5	momo_gero	momogero@gmail.com	Momo123!	1
Editar Copiar Borrar	6	brunenger_ok	brunengerok@gmail.com	Brunenger123!	1
Editar Copiar Borrar	7	pimpeano_in	pimpeanoin@gmail.com	Pimpe123!	1

```
SELECT * FROM `pasajero`
```

Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear código PHP] [Actualizar]

IDPasajero	nombre_pasajero	apellido_pasajero	dni_pasajero	telefono	fecha_nacimiento	nro_pasaporte	pais_emision	vencimiento_pasaporte	usuario_id

	IDEmpleado	nombre_empleado	apellido_empleado	dni_empleado	sueldo	departamento_id	aeropuerto_id	usuario_id
Editar Copiar Borrar	1	Lautaro	del Campo	40555444	3500.00	2	5	2
Editar Copiar Borrar	2	David	Quintanilla	40222333	4000.00	1	5	3
Editar Copiar Borrar	3	Gonzalo	Banzas	38444555	1800.00	2	1	4
Editar Copiar Borrar	4	Gerónimo	Benavides	37555666	1500.00	1	3	5
Editar Copiar Borrar	5	Bruno	Kr Kr Kr	41666777	1500.00	3	3	6
Editar Copiar Borrar	6	Lautaro	Pimpeano	40111222	5500.00	1	5	7

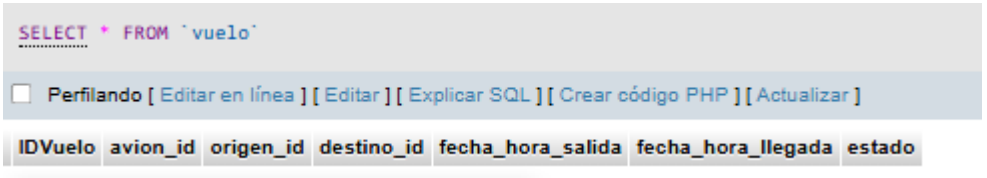
La tabla Asiento representa los asientos disponibles en cada avión y permite registrar su disponibilidad. La tabla Vuelo almacena los vuelos programados, relacionándolos con el avión utilizado y los aeropuertos de origen y destino, además de registrar sus horarios y estados.

	IDAsiento	codigo	avion_id	tipo	disponible
Editar Copiar Borrar	AV-A320-01-14C	14C	AV-A320-01	Pasillo	1
Editar Copiar Borrar	AV-A320-01-14D	14D	AV-A320-01	Pasillo	1
Editar Copiar Borrar	AV-A320-01-14E	14E	AV-A320-01	Medio	1
Editar Copiar Borrar	AV-A320-01-14F	14F	AV-A320-01	Ventana	1
Editar Copiar Borrar	AV-A320-01-15A	15A	AV-A320-01	Ventana	1
Editar Copiar Borrar	AV-A320-01-15B	15B	AV-A320-01	Medio	1
Editar Copiar Borrar	AV-A320-01-15C	15C	AV-A320-01	Pasillo	1
Editar Copiar Borrar	AV-A320-01-15D	15D	AV-A320-01	Pasillo	1
Editar Copiar Borrar	AV-A320-01-15E	15E	AV-A320-01	Medio	1
Editar Copiar Borrar	AV-A320-01-15F	15F	AV-A320-01	Ventana	1
Editar Copiar Borrar	AV-A320-01-16A	16A	AV-A320-01	Ventana	1
Editar Copiar Borrar	AV-A320-01-16B	16B	AV-A320-01	Medio	1
Editar Copiar Borrar	AV-A320-01-16C	16C	AV-A320-01	Pasillo	1
Editar Copiar Borrar	AV-A320-01-16D	16D	AV-A320-01	Pasillo	1
Editar Copiar Borrar	AV-A320-01-16E	16E	AV-A320-01	Medio	1
Editar Copiar Borrar	AV-A320-01-16F	16F	AV-A320-01	Ventana	1
Editar Copiar Borrar	AV-A320-01-17A	17A	AV-A320-01	Ventana	1
Editar Copiar Borrar	AV-A320-01-17B	17B	AV-A320-01	Medio	1
Editar Copiar Borrar	AV-A320-01-17C	17C	AV-A320-01	Pasillo	1
Editar Copiar Borrar	AV-A320-01-17D	17D	AV-A320-01	Pasillo	1
Editar Copiar Borrar	AV-A320-01-17E	17E	AV-A320-01	Medio	1
Editar Copiar Borrar	AV-A320-01-17F	17F	AV-A320-01	Ventana	1
Editar Copiar Borrar	AV-A320-01-18A	18A	AV-A320-01	Ventana	1
Editar Copiar Borrar	AV-A320-01-18B	18B	AV-A320-01	Medio	1

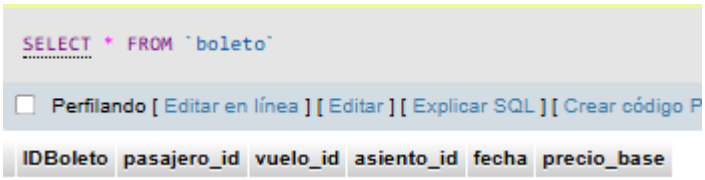
Seleccionar todo *Para los elementos que están marcados:* Editar Copiar Borrar Exportar

<< < 2 > >> Número de filas: 25 Filtrar filas: Buscar en esta tabla Ordenar según la clave: Ninguna

Consola



La tabla Boleto vincula a cada pasajero con un vuelo y un asiento, registrando también la fecha y el precio base. La tabla intermedia Boleto_Servicio permite asociar varios servicios adicionales a un mismo boleto. De esta manera, un pasajero puede contratar diferentes servicios sin necesidad de crear una tabla independiente para cada combinación.





La tabla Pago registra los pagos correspondientes a los boletos, incluyendo los costos de los servicios y el importe total. Finalmente, CheckIn permite registrar la información del proceso de registro previo al embarque, relacionando los vuelos y los pagos con la puerta de embarque correspondiente.

TABLA DE PAGO

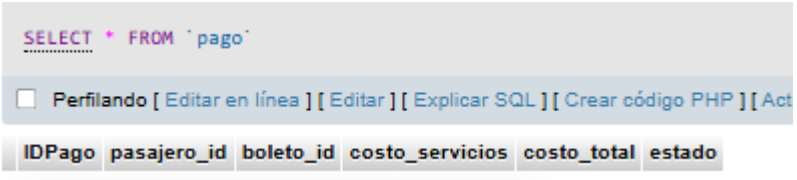
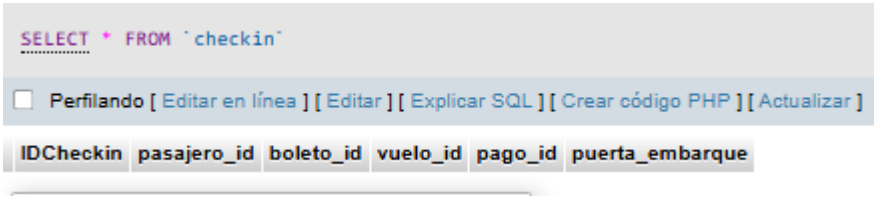


TABLA DE PAGO

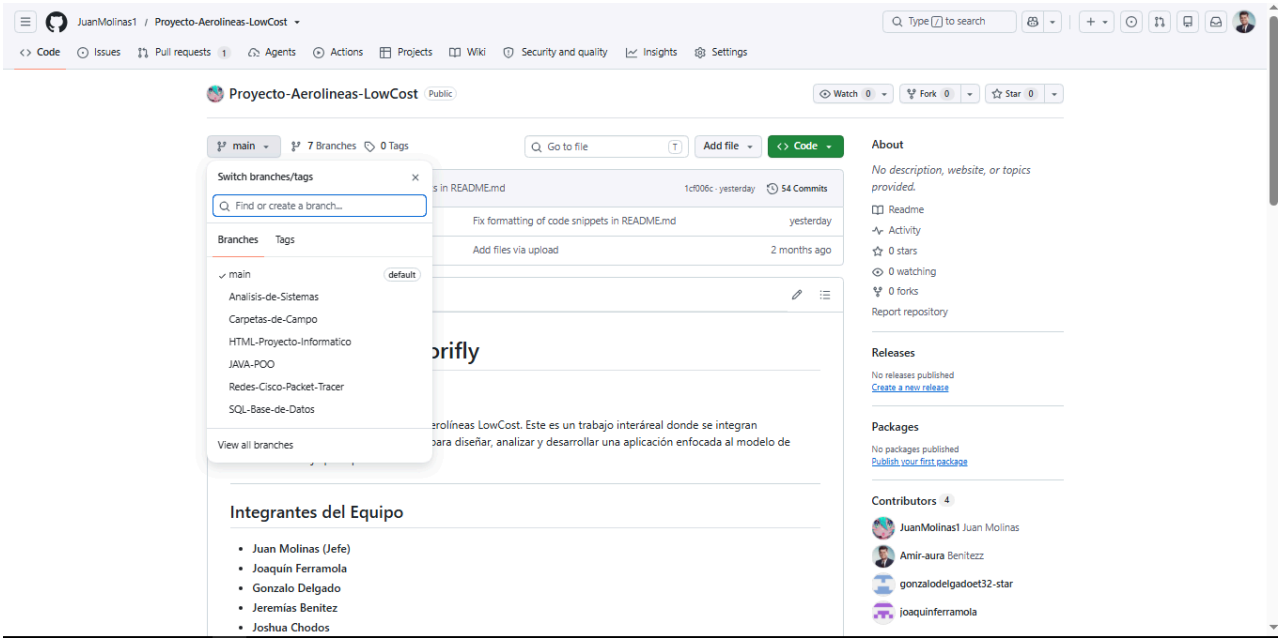


Todas estas tablas se comunican mediante un esquema de claves primarias y foráneas.

Con este diseño, logramos estructurar de punta a punta la operativa de la aerolínea, cubriendo desde la gestión de recursos humanos y tecnológicos hasta la comercialización de pasajes, cobros y el flujo de embarque. A lo largo del proyecto, iteré esta estructura hasta consolidar una versión final enriquecida con restricciones avanzadas y relaciones optimizadas.

Asimismo, como parte de los avances documentales y técnicos de la jornada, subimos a la carpeta de campo grupal los registros correspondientes a los diagramas de Gantt y PERT, junto con el análisis FODA, y dimos el banderazo de salida al desarrollo de la aplicación con la creación formal del primer archivo .java.

Por último, en GitHub se crearon las branches correspondientes de cada materia y se comenzó a organizar el repositorio con mayor precisión, debido al creciente número de archivos incorporados al proyecto.



README

Main - Proyecto Chorifly

Temática: Aerolíneas Low-Cost

Bienvenido al repositorio del proyecto Aerolíneas LowCost. Este es un trabajo interáreal donde se integran conocimientos de múltiples asignaturas para diseñar, analizar y desarrollar una aplicación enfocada al modelo de aerolíneas de bajo presupuesto.

Integrantes del Equipo

- Juan Molinas (Jefe)
- Joaquín Ferramola
- Gonzalo Delgado
- Jeremías Benitez
- Joshua Chodos

Materias Involucradas

En esta sección se detalla el trabajo realizado para cada asignatura y el docente a cargo de la evaluación:

Programación Orientada a Objetos (POO) (Prof. Nahuel Acosta)

- **Descripcion del trabajo:** Con esta materia nos encargamos principalmente de las vistas del empleado, con el fin de poder hacer toda la administración de la aerolínea. Más información en JAVA - POO.

Redes (Prof. Nahuel Acosta)

- **Descripcion del trabajo:** Con esta materia nos encargamos principalmente de toda la topología y distribución de los dispositivos según las areas y su importancia. Más información en Redes - Cisco Packet Tracer.

Fecha 27/08/2026

Hoy junto a mi compañero Joaquín Ferramola estuvimos investigando sobre la Aerolinea Low Cost.

Hicimos una lista de sus atributos en general, tanto como positivos y/o regulares sobre ello, para luego poder separar para el análisis FODA, que se separa por; Fortalezas, Oportunidades,Debilidades y Amenazas.

A continuación adjunto el link: FODA analisis

Y su transcripción a este documento:

Analisis FODA
(Aerolínea Low Cost)

-Costos baratos
-Personal entrenado (pilotos reconocidos y pilotos altamente capacitados)
-Viajes cortos/rápidos
-Salario muy estable
-Equipaje limitado
-Servicios acotados (no comida/bebida)
-Pago extra por dichos servicios
-Uso solo 1 o 2 modelos de aviones
-Las aerolineas mas equipadas se adaptaron creando paquetes “economicos” para asemejarse a las low cost
-No pueden realizar viajes muy largos debido al presupuesto ya planeado de la nafta
-Los precios por los pilotos han subido, se acaba la mano de obra barata para las low cost
-Paises (como francia) reemplazan los viajes cortos con trenes de alta velocidad
-Nueva tecnologia sobre aviones de un pasillo unico, mas pequeños y faciles de mantener, recorren mas distancias
-Las empresas independientes o pequeñas ya no pueden costear viajes lujosos o de aerolineas comunes, asi que estan prefiriendo las low cost
-Lugares como america latina o asia toman colectivos para viajes de 20 o 30 horas para llegar a ciudades del interior, las low cost estan tomando esto como una oportunidad

Fortaleza
- Costos baratos
- Personal entrenado (pilotos reconocidos y pilotos altamente capacitados)
- Viajes cortos/rápidos
- Salario muy estable
- Uso solo 1 o 2 modelos de aviones

Oportunidad
- Nueva tecnología sobre aviones de un pasillo único, mas pequeños y fáciles de mantener, recorrer más distancias
- Las empresas independientes o pequeñas ya no pueden costear viajes lujosos o de aerolineas comunes, asi que estan prefiriendo las low cost
- Lugares como américa latina o asia toman colectivos para viajes de 20 o 30 horas para llegar a ciudades del interior, las low cost están tomando esto como una oportunidad

Debilidad
- Equipaje limitado
- Países (como francia) reemplazan los viajes cortos con trenes de alta velocidad
- Pago extra por dichos servicios
- No pueden realizar viajes muy largos debido al presupuesto ya planeado de la nafta
- Servicios acotados (no comida/bebida)

Amenaza
- Las aerolíneas más equipadas se adaptaron creando paquetes “economicos” para asemejarse a las low cost
- Los precios por los pilotos han subido, se acaba la mano de obra barata para las low cost

Fecha 31/08/2026

El día de hoy se subió a la carpeta de campo grupal los registros correspondientes al diagrama de Gantt, el diagrama PERT y el análisis FODA. Se creó el primer archivo `.java`, dando inicio al desarrollo de la aplicación.

Se investigaron y definieron los materiales que se utilizarían para construir la red en Cisco Packet Tracer. Se seleccionaron los routers ISR 4331, los switches centrales Catalyst 3650-24PS, los switches de acceso Catalyst 2960 y el servidor Server-PT.

Los mismos serán usados para la creación del presupuesto.

Además, comenzamos a desarrollar el primer diseño de la página web de Chorifly utilizando HTML y CSS, sin incorporar JavaScript por el momento. Se diseñaron elementos iniciales

como el encabezado, el menú de navegación, el calendario, la sección principal y el pie de página. Sin embargo, el desarrollo web quedó en espera por que pense que era mas importante la parte de base de datos.

Fecha 2/09/26:

En Base de Datos se realizaron modificaciones estructurales en el modelo de base de datos original. Se incorporó el campo aeropuerto_id en la tabla de empleados con el fin de identificar el aeropuerto al que pertenece cada uno. Asimismo, se agregó el campo usuario_id en las tablas de pasajeros y empleados para vincularlos con las cuentas de usuario del sistema. Esta relación permite valores nulos, pero limita a un único usuario la asociación de cada pasajero o empleado.

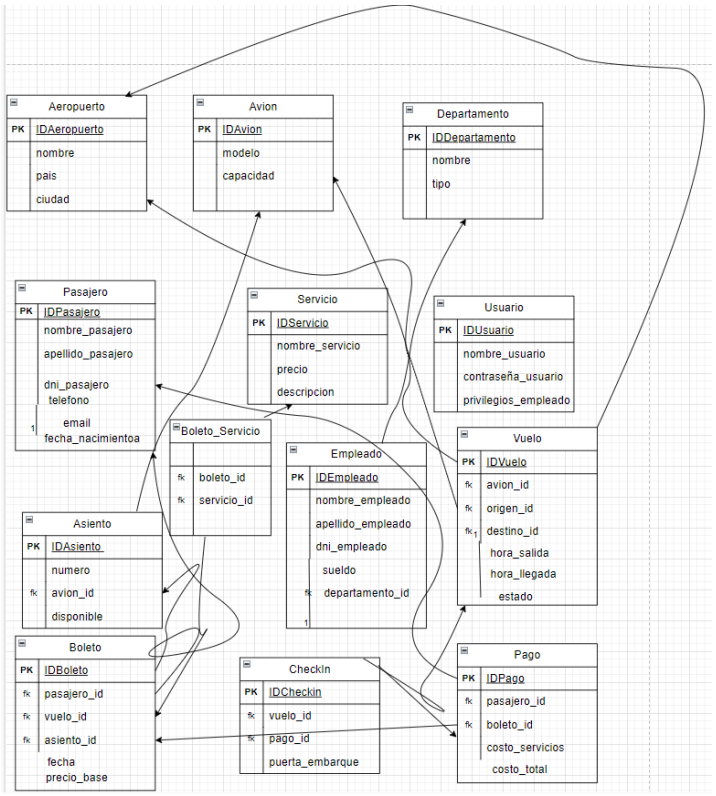
En Redes Se subió al repositorio de GitHub una primera versión preliminar de la topología de red desarrollada en Cisco Packet Tracer. Aunque el diseño todavía se encontraba incompleto, permitió sentar las bases iniciales del esquema de conectividad del proyecto.

Con el objetivo de optimizar la organización del repositorio y facilitar la colaboración, se comenzó a implementar de manera uniforme la creación de archivos README en todas las branches. Esto permite documentar claramente el contenido de cada una y entender la distribución de los archivos correspondientes a las distintas materias.

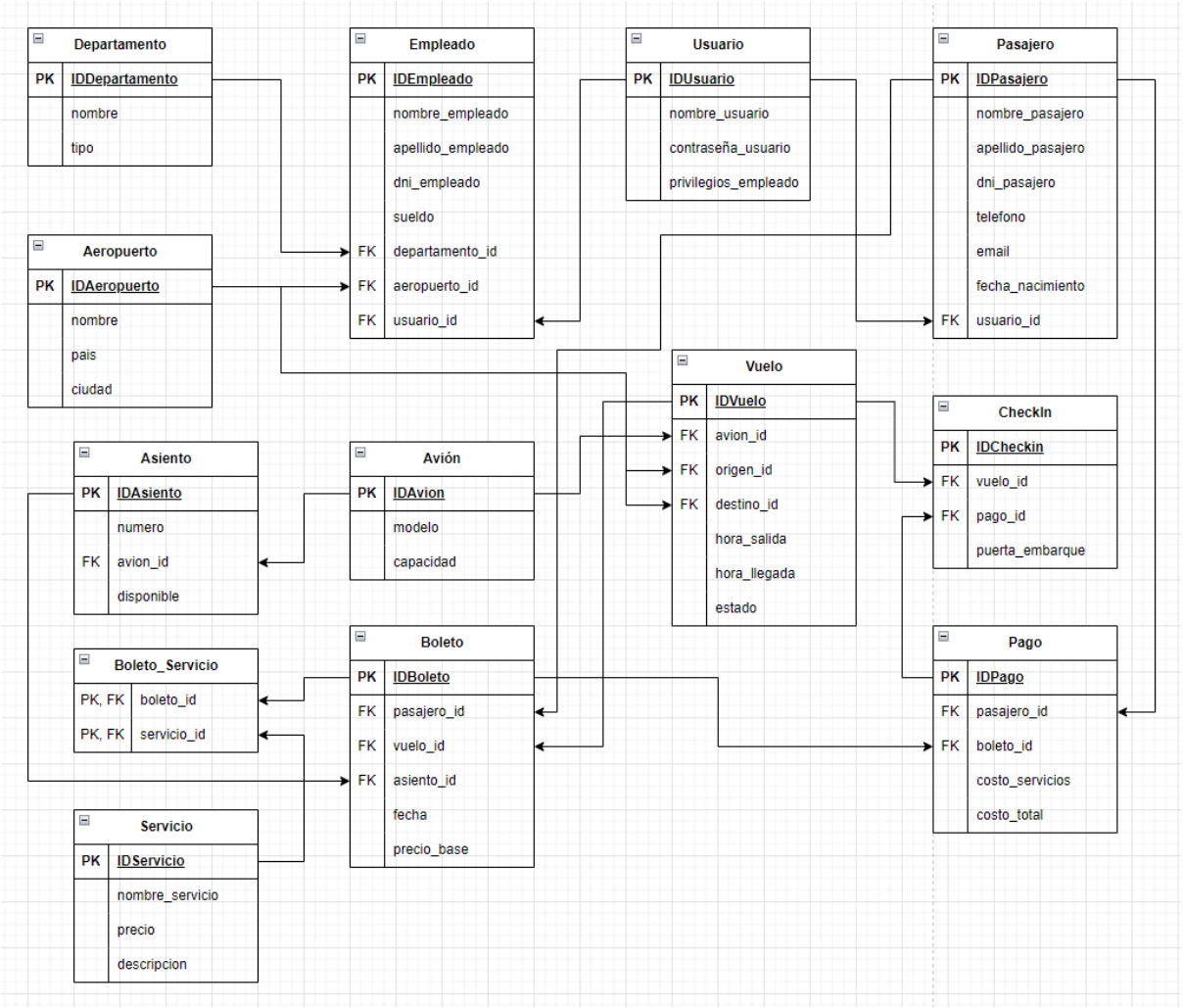
En el desarrollo del CRUD, se creó un archivo que enlaza a un documento de Google donde se definió el CRUD inicial del proyecto.

En la Reestructuración del Diagrama Entidad-Relación (DER) Se evaluó el Diagrama Entidad-Relación (DER) presentado por Joshua Chodos, detectando serias fallas de prolijidad, sintaxis y distribución visual. Ante la imposibilidad de corregirlo sobre la marcha, se rehizo el diagrama por completo desde cero para obtener una arquitectura de datos

ordenada, comprensible y fiel a las entidades del sistema, que sirva de cimiento para la base de datos (diseño original por Joshua Chodos).



y tras una revision como global por parte del grupo, junto a juan molinas y joaquin ferramola rehicimos el diagrama de entidad relación para que sea mas legible y que mucho mas prolijo que ese diagrama todo desordenado.(nuestro diseño)



07/09/2026

Ausencia y Avances en el Repositorio, Debido a una mudanza personal, no pude asistir presencialmente a la jornada, pero me comuniqué con mis compañeros para ponerme al tanto de los cambios realizados en el repositorio de GitHub. Durante ese día se crearon los archivos README tanto para la rama principal (*main*) como para las distintas *branches*, utilizando Markdown para estructurar y presentar la información con claridad. El README principal se organizó con títulos y subtítulos para detallar la temática del proyecto, los integrantes, las materias, la descripción general de Chorifly, los objetivos y el registro de avances, incluyendo enlaces directos a cada rama. Asimismo, los README específicos se adaptaron a cada asignatura mediante encabezados, listados de archivos y descripciones breves, además de

enlaces de navegación bidireccional; por ejemplo, en Análisis de Sistemas se documentaron los archivos relativos al FODA, diagramas de Gantt y PERT, MVP, requisitos, Scrum y análisis del proyecto. Esta estructuración permitió centralizar la información y facilitar la navegación, concentrándose la jornada exclusivamente en la organización del repositorio sin registrar otros avances en el desarrollo del sistema.

Carga de Datos Iniciales en la Base de Datos (15/09/2026) Fecha: 15 de septiembre de 2026. Se elaboraron las sentencias INSERT necesarias para poblar la base de datos con información inicial, configurando un escenario de pruebas más completo. Se incorporaron registros para las tablas de departamentos, aeropuertos, aviones, servicios, usuarios, empleados y pasajeros, así como datos correspondientes a asientos, vuelos, boletos, servicios contratados, pagos y check-in. Entre estos registros se incluyeron datos ficticios con nombres personalizados para otorgar una identidad propia al proyecto, con el objetivo de comprobar las relaciones entre tablas y probar funcionalidades sin requerir cargas manuales repetitivas. Se trató de una carga preliminar y flexible, sujeta a modificaciones o depuraciones posteriores a medida que el desarrollo del software avanzara.

Modelado y Distribución de Asientos Fecha: 15 de septiembre de 2026. Se inició el desarrollo específico de la distribución de asientos de los aviones mediante la creación de un archivo SQL enfocado en una única aeronave. El esquema planteó filas de seis asientos identificados de la A a la F, distinguiendo las ubicaciones de ventana, medio y pasillo, y dotando a cada elemento de un identificador, código, referencia de avión, tipo y estado de disponibilidad. En esta versión inicial se generaron 31 filas que totalizaron 186 lugares por avión, configurados inicialmente como disponibles para ser empleados en las primeras pruebas de selección y representación gráfica desde la aplicación en Java.

Al analizar cómo se integraría este modelo con la aplicación desarrollada en Java, se detectó una dificultad lógica importante vinculada con la relación entre los asientos y los vuelos. Dado que los asientos se encontraban asociados de forma directa al avión y no a un vuelo específico en particular, permitir que una misma aeronave tuviera varios vuelos programados al mismo tiempo generaba inevitables conflictos en la disponibilidad de los lugares. Por ejemplo, dos vuelos diferentes asignados a una misma unidad consultarían el mismo conjunto físico de asientos, lo que imposibilitaba determinar con precisión cuáles se encontraban realmente libres para cada operación comercial. Con el propósito de resolver este inconveniente estructural, se estableció como regla de negocio que cada avión pudiera tener, como límite máximo, un único vuelo en estado "Programado" de manera simultánea. Posteriormente, este estado se incorporó formalmente al conjunto de estados posibles dentro del ciclo de vida de los vuelos, logrando que la programación operativa respetara de manera estricta la lógica definida para la asignación de plazas.

A partir de estas revisiones técnicas y de diseño, también se tomó la decisión de reducir la cantidad total de asientos por aeronave de 186 a 180 plazas. Esta modificación obedeció a la necesidad de facilitar la representación gráfica de los lugares dentro de la interfaz en Java, ya que trabajar con una cifra divisible exacta en 30 filas de seis asientos simplificaba notablemente la distribución visual de los botones, la disposición de los espacios y la organización general de la ventana de selección de pasajeros. De esta manera, el modelo físico se adaptaba de forma óptima a los requerimientos estéticos y funcionales de la interfaz gráfica proyectada.

Junto con este cambio en la capacidad, se revisó detalladamente el formato de identificación asignado a cada asiento. En una primera instancia se utilizaban nomenclaturas simples como 1A, 1B o 1C, donde el número representaba la fila y la letra indicaba la posición exacta. Sin

embargo, se optó por migrar hacia un formato estandarizado de dos cifras para mantener una nomenclatura completamente uniforme, por lo que un código como 1A pasó a denominarse formalmente 01A. Para implementar esta modificación a nivel relacional, la columna número, que originalmente se había concebido como un tipo de dato entero (INT), fue reemplazada por un campo denominado código bajo el tipo de dato texto (VARCHAR), adquiriendo la capacidad de almacenar la identificación completa del asiento en un único campo unificado. Así, el esquema evolucionó hacia códigos normalizados como 01A, 02B o 30F, agilizando considerablemente su lectura, las consultas de búsqueda y su posterior manipulación desde la aplicación en Java.

En conjunto, todas estas evoluciones permitieron no solo preparar de manera robusta los datos indispensables para las primeras pruebas del sistema, sino también replantear decisiones fundamentales de diseño que repercutían directamente sobre la estabilidad del software. La carga inicial proveyó los registros necesarios para verificar con éxito las asociaciones entre las entidades, mientras que el refinamiento del modelo de asientos permitió anticipar con precisión los requerimientos de la interfaz gráfica y detectar restricciones críticas que debían resolverse de forma anticipada. De este modo, la etapa representó un progreso sustancial tanto en la estructuración de la base de datos relacional como en la definición de los lineamientos lógicos que conectarían el almacenamiento con la arquitectura del sistema.

Fecha: 16 /9/2026

Durante esta jornada de trabajo se continuó operando de manera intensiva sobre la base de datos del proyecto Chorifly. Tras haber preparado e incorporado las sentencias INSERT indispensables para poblar los registros iniciales y verificar la integridad de las tablas y sus relaciones, se ejecutó una minuciosa tarea de depuración y limpieza de los datos de prueba.

Luego de analizar detenidamente el contenido que se había estructurado previamente, se tomó la determinación de eliminar los comandos correspondientes a tablas que ya no se consideraban relevantes ni esenciales para el estado actual del desarrollo del software. El propósito principal de esta acción consistió en prevenir la acumulación innecesaria de información marginal y conservar exclusivamente aquellos datos que aportaran valor operativo directo para las pruebas funcionales de la aplicación. De esta forma, la base de datos quedó perfectamente sincronizada con los requerimientos reales de la etapa, dejando temporalmente programada para fases posteriores la creación de vistas avanzadas, activadores (triggers) y procedimientos almacenados.

Paralelamente, se completó la organización integral del repositorio oficial en GitHub, estructurando adecuadamente las distintas ramas (branches) correspondientes a cada materia, sus directorios específicos y sus respectivos archivos README.md. Esta medida organizativa garantizó una arquitectura de archivos limpia y transparente, facilitando notablemente el acceso a los materiales teóricos y prácticos del proyecto académico.

En el área de Redes e Infraestructura, colaborando conjuntamente con Joaquín Ferramola, se concluyó el diseño topológico de las 20 subredes planificadas en Cisco Packet Tracer, configurando operativamente los cuatro servidores principales del entorno: DHCP, DNS, HTTP y FTP. No obstante, aún quedaba pendiente consolidar la interconexión completa entre todas las subredes. Para superar este desafío técnico, se diagramó la incorporación estratégica de módulos de expansión serial tipo NIM-2T en los routers Cisco ISR 4331, con la finalidad específica de habilitar interfaces seriales físicas y ejecutar con éxito el enrutamiento estático entre los distintos segmentos de red.

Por otro lado, trabajando codo a codo con Gonzalo Delgado, se iniciaron los bocetos conceptuales y el diseño preliminar de las primeras interfaces visuales de la aplicación Java

mediante el uso de componentes JFrame. Durante este tramo inicial del desarrollo frontend, también comenzaron a delinearse los criterios arquitectónicos sobre dónde y de qué manera se aplicarían las validaciones y los controles de integridad del sistema.

Finalmente, tras una evaluación conjunta, el diseño prototípico inicial de las ventanas fue descartado por completo para ser reemplazado por una nueva propuesta estética mucho más ordenada, limpia y visualmente atractiva. Este rediseño conservó íntegramente los campos operativos y las funcionalidades previamente especificadas, pero introdujo una presentación mejorada y un uso estratégico de la paleta de colores para simplificar la identificación de los diferentes elementos interactivos dentro de la interfaz de usuario.

Fecha: 18 de septiembre de 2026

En el transcurso de esta jornada se introdujo una modificación de gran relevancia conceptual sobre el Producto Mínimo Viable (MVP) del sistema Chorifly. Se determinó transformar radicalmente el enfoque con el que se administraba el equipaje adicional, suprimiendo por completo la compleja alternativa de calcular su valor tarifario en función estricta del peso acumulado. Como contrapartida, se resolvió que el costo del equipaje extra pasaría a fijarse exclusivamente a través de los servicios complementarios ofrecidos dentro del catálogo del sistema. Bajo este nuevo paradigma, si un pasajero omite la contratación previa del servicio de equipaje extra, quedará inhabilitado para transportar bultos adicionales. Esta importante decisión arquitectónica simplificó de manera notable la lógica de programación requerida, eliminando algoritmos de cálculo basados en variaciones de peso y centralizando la imputación económica de forma directa en el servicio asociado. De este modo, el equipaje adicional dejó de concebirse como una propiedad independiente del pasajero para convertirse estrictamente en una prestación derivada de la contratación de un servicio.

En lo concerniente al modelo de base de datos, se aplicaron múltiples ajustes orientados a subsanar omisiones previas y elevar los estándares de validación de los datos. Se incorporó formalmente el campo email en la tabla Usuario, un atributo indispensable para registrar el correo electrónico de cada cuenta que se había pasado por alto involuntariamente en el modelado inicial. Como consecuencia directa de esta incorporación estructural, se actualizaron todas las sentencias INSERT precedentes para que reflejaran fielmente el nuevo campo y mantuvieran plena coherencia con la versión vigente de la tabla.

Asimismo, se modificó el tipo de datos asignado a la columna correspondiente a la fecha de nacimiento, transformándola de DATETIME a un formato estrictamente DATE, dado que para registrar un nacimiento carecía por completo de sustento técnico almacenar una marca de tiempo horaria. En la misma línea de optimización, se implementó de forma masiva la restricción de obligatoriedad NOT NULL en prácticamente la totalidad de las columnas del esquema relacional, con el propósito fundamental de bloquear el ingreso de valores vacíos en campos que exigen información mandatoria. De esta manera, si desde la capa de la aplicación en Java se intentara persistir información anómala o incompleta, la base de datos reaccionaría disparando una excepción controlable mediante el manejo de SQLException, permitiendo identificar con absoluta precisión el origen del fallo consultando el código de error correspondiente. Esta estrategia se diseñó para agilizar considerablemente el diagnóstico y la depuración de errores durante el ciclo de desarrollo del software.

Complementariamente, se dio de alta un nuevo departamento institucional vinculado directamente con la gestión de equipajes. Este sector se añadió como una alternativa funcional más dentro del organigrama de departamentos disponibles, sin que fuera imperativo desplegar lógica operativa específica para este módulo en esta fase temprana del proyecto. En la misma sintonía de normalización, se renombraron los campos temporales de la tabla Vuelo:

las columnas identificadas originalmente como hora_salida y hora_llegada pasaron a denominarse formalmente fecha_hora_salida y fecha_hora_llegada, respectivamente. Este ajuste léxico buscó reflejar con absoluta claridad que ambos atributos almacenan de manera integrada tanto la fecha calendario como el horario exacto de cada operación de vuelo.

En paralelo, se dio inicio al desarrollo de la arquitectura base del sistema en Java utilizando el entorno de desarrollo NetBeans, estructurando rigurosamente el código fuente bajo el paquete lógico denominado chorifly. El objetivo central de esta etapa consistió en maquetar el mapeo de clases orientado a objetos indispensable para representar los componentes neurálgicos del sistema y consolidar la estructura del modelo de datos orientado a objetos (POJO), sentando las bases necesarias para transferir posteriormente este desarrollo a Gonzalo Delgado, quien continuaría con la implementación de las vistas y el diseño gráfico de las pantallas.

Las primeras clases de entidades codificadas en el repositorio fuente fueron Aeropuerto, Avion, Asiento, Departamento, Persona, Empleado, Pasajero y Usuario, complementadas por la clase controladora central llamada Aerolinea y la clase ejecutable principal Chorifly. La premisa fundamental consistió en emular fielmente las entidades relacionales de la base de datos mediante instancias de objetos en Java, garantizando que un pasajero pudiera manipularse operativamente como un objeto de la clase Pasajero para ejecutar sobre él operaciones de persistencia. Durante esta fase inicial, las clases se enfocaron exclusivamente en la representación estructural de los atributos privados y sus respectivos métodos de acceso encapsulados (getters y setters), omitiendo deliberadamente la lógica de las operaciones CRUD y los patrones de acceso a datos DAO. Para jerarquizar correctamente la relación entre las personas registradas en el sistema, se programó inicialmente una clase abstracta denominada Persona, de la cual heredan mediante la directiva extends las clases derivadas Empleado y Pasajero. Este diseño de herencia permitió unificar atributos de identidad

comunes y segregar al mismo tiempo las propiedades particulares de cada rol. Posteriormente se desarrolló la clase Departamento, configurada con el nombre descriptivo y un atributo lógico o enumerado capaz de determinar si el área pertenece al sector aeronáutico o terrestre.

A continuación, se avanzó sobre la implementación de la clase Aeropuerto, la cual simboliza una sede operativa de la aerolínea e integra datos geográficos de ubicación como país y ciudad, además de una estructura de colección orientada a albergar a su personal asignado. Se proyectó una relación bidireccional plenamente operativa entre las clases Aeropuerto y Empleado: el aeropuerto conserva una lista referencial de los trabajadores vinculados a su sede, mientras que cada empleado identifica explícitamente el aeropuerto al cual presta servicio. Si bien en un análisis preliminar se evaluó la posibilidad de que los comandantes o pilotos operaran sin un aeropuerto de adscripción fija —permitiendo que dicho atributo admitiera valores nulos y se omitiera en el constructor de Empleado—, dicha postura técnica fue descartada con posterioridad, resolviéndose de manera uniforme que la totalidad de los empleados debían poseer obligatoriamente una asignación departamental y de sede equiparada. Por consiguiente, la excepción planteada para los pilotos no formó parte del diseño definitivo.

Asimismo, se estructuró formalmente la relación composicional entre las clases Avion y Asiento. Se definió que una aeronave contiene una colección de múltiples asientos y que cada asiento pertenece de manera exclusiva a un único avión, materializando una relación de cardinalidad uno a muchos (1-N) de carácter bidireccional dentro del código fuente. Esto implica que el objeto Avion aloja una lista estructurada con sus asientos, al tiempo que cada instancia de Asiento referencia de manera inequívoca al avión que integra. Adicionalmente, se ideó un formato de identificación descriptivo que facilitara el reconocimiento visual de los elementos sin depender exclusivamente de claves numéricas autoincreméntales

convencionales; dicho patrón combinaba el identificador del avión con el código unívoco del asiento, estructurándose bajo denominaciones como AV-A320-02-1A. Pese a haber definido rigurosamente esta arquitectura de identificadores compuestos, se estipula que los mismos no se integrarían dentro de los constructores primitivos de las clases Avion y Asiento, puesto que la generación de dichos registros corresponderá a procesos lógicos externos a este módulo en particular.

Finalmente, todas las clases desarrolladas hasta este punto del ciclo de vida del proyecto se encuentran provistas exclusivamente de sus atributos privados, constructores base y métodos de acceso normalizados. El esfuerzo técnico de la jornada culminó con un éxito rotundo al consolidar una arquitectura orientada a objetos sólida, limpia, mantenible y rigurosamente alineada con el esquema relacional de la base de datos, dejando preparado el terreno fundacional sobre el cual se edificarán las capas de conectividad, los controladores lógicos y, subsecuentemente, las interfaces gráficas definitivas del sistema.

Fecha: 23/09/26

En el transcurso de esta jornada se dio inicio formal al desarrollo y estructuración integral de los presupuestos de infraestructura de red para el proyecto Chorifly, un proceso analítico y de cálculo que demandó un trabajo sostenido hasta su culminación definitiva el 30 de septiembre de 2026. Este bloque de tareas se centró en dimensionar de manera precisa los costos, los requerimientos de hardware, el cableado estructurado y la distribución de dispositivos de conectividad necesarios para sostener operativamente los 20 sectores funcionales que componen la arquitectura de la aerolínea.

Durante la primera etapa de este periodo, comprendida entre el 21 y el 25 de septiembre, se establecieron las bases de cálculo y los criterios de asignación tecnológica por área. Se analizaron detalladamente las necesidades específicas de cada departamento en función de su

criticidad operativa, la cantidad de puestos informáticos requeridos y la complejidad de sus enlaces de red. Para ello, se estructuró un modelo económico en pesos argentinos (ARS) que contempló la integración de terminales de PC, tendidos de cables, switches de acceso, switches multicapa y routers de distribución para cada división de la compañía, abarcando desde áreas administrativas y comerciales hasta sectores altamente sensibles como la seguridad aeroportuaria, el control de tráfico aéreo y la gestión de crisis.

Conforme avanzaba la semana y se aproximaba el cierre del mes, las jornadas del 28 al 30 de septiembre se destinaron a consolidar, depurar y balancear los totales presupuestarios. El cálculo global arrojó un dimensionamiento total de 1.200 computadoras distribuidas en los diferentes sectores, respaldadas por una infraestructura de red compuesta por 1.412 tramos de cable, 20 switches multicapa de alta capacidad, 20 routers de enrutamiento y 108 switches de acceso perimetral. Todo este despliegue técnico derivó en un presupuesto de infraestructura consolidado que ascendió a un total de 485.783.000 ARS.

Asimismo, en esta fase final se clasificaron los sectores según su nivel de prioridad y equipamiento tecnológico —agrupándolos en categorías básicas, medias y altas en función de la inversión por puesto de trabajo—, destacándose áreas de máxima criticidad como la Gestión de Crisis (que concentró el 15,67% del presupuesto total con 76.125.000 ARS) y la Seguridad Aeroportuaria (con un 11,24% y 54.590.000 ARS). De este modo, el trabajo finalizado el 30 de septiembre no solo permitió cuantificar con exactitud la inversión económica requerida para el deployment tecnológico de Chorifly, sino que también aportó una base sólida, coherente y detallada para la integración de la infraestructura física con el diseño de red previamente modelado en Cisco Packet Tracer.

Fecha: 21/09/26 al 30/09/26

Durante el período comprendido entre el 21/09/26 y el 30/09/26, en paralelo con el trabajo de los presupuestos de infraestructura de red, me dediqué a revisar, ordenar y explicar con el mayor detalle posible los 50 requisitos del sistema Chorifly que habíamos validado el 19/08/26: 25 funcionales y 25 no funcionales. Los dejé redactados en dos documentos PDF, «Chorifly - Requisitos Funcionales» y «Chorifly - Requisitos no Funcionales», y en cada uno de los requisitos mantuve la misma estructura: primero el enunciado técnico de lo que el sistema debe hacer o cumplir, y a continuación una aclaración, en lenguaje llano, de lo que eso significa en la práctica. Elegí este formato porque entendí que un requisito escrito solo en términos técnicos es difícil de defender en una exposición y difícil de verificar durante las pruebas de QA, mientras que con una explicación clara cualquier compañero o profesor puede comprobar si realmente se cumplió. En esta entrada dejo registrado todo lo que desarrollé, agrupado por temas, junto con las decisiones de diseño que hay detrás de cada grupo de requisitos.

Requisitos funcionales

Los requisitos funcionales describen qué es lo que el sistema hace. Los 25 que redacté los organicé en seis bloques que siguen el recorrido real de una aerolínea low cost: la gestión de los vuelos, el acceso y las cuentas de usuario, la venta de pasajes, la distribución de asientos, el check-in y el embarque, y por último los pagos y la seguridad de los datos. Mi criterio fue que cada requisito se pudiera comprobar con una prueba concreta: se hace una acción en la aplicación y se observa un resultado que se cumple o no se cumple.

Gestión de vuelos (requisitos 1 a 7)

Los tres primeros requisitos cubren el ciclo de vida básico de un vuelo. El requisito 1, registrar vuelos, establece que el administrador puede dar de alta nuevos itinerarios y rutas aéreas, guardando en la base de datos la fecha, la hora, el avión y los destinos de cada viaje

nuevo. El requisito 2, modificar vuelos, permite editar un itinerario ya cargado, ya que en la actividad aérea es normal tener que reprogramar un horario o cambiar el avión asignado ante un imprevisto. El requisito 3, eliminar vuelos, contempla la baja o cancelación de un trayecto: lo pensé de modo que el operador pueda retirarlo de la oferta comercial o marcarlo como inactivo, sin destruir el registro de que ese vuelo existió.

Los requisitos 4, 5 y 6 son reglas de negocio que quise que controlara el backend y no solamente la pantalla, porque una regla que vive solo en la interfaz se puede esquivar y una regla que vive en el backend se cumple siempre. El requisito 4, validar la cronología de fechas, bloquea el registro de un vuelo cuya fecha de salida sea posterior a la de llegada; el backend verifica de forma matemática que el tiempo sea coherente y así se evitan errores de carga manual. Este requisito está directamente relacionado con el cambio que hice en la base de datos de renombrar hora_salida y hora_llegada como fecha_hora_salida y fecha_hora_llegada, porque ahora cada columna guarda la fecha y la hora juntas y la comparación se puede hacer con precisión.

El requisito 5, restringir vuelos simultáneos por aeronave, impide que un mismo avión tenga más de un vuelo en estado «Programado» al mismo tiempo. Esta regla nació de un problema real que detecté al diseñar la distribución de asientos: como los asientos pertenecen al avión y no a un vuelo en particular, si una misma aeronave tuviera dos vuelos programados a la vez, ambos consultarían el mismo conjunto físico de asientos y sería imposible saber cuáles están libres para cada operación. Al limitar a un solo vuelo programado por avión, la disponibilidad de los lugares queda siempre bien definida, y además se evita la superposición horaria del uso de una misma unidad física, que en la vida real sería imposible.

El requisito 6, bloqueo automático de rutas largas, hace que el backend impida asignar tramos mayores a 5 horas. Con esto el software fuerza el cumplimiento del modelo operativo low

cost, que se basa en rutas cortas «en espejo», es decir, trayectos de ida y vuelta breves que permiten rotar el mismo avión muchas veces por día. Esta decisión también se apoya en el análisis FODA que hicimos, donde figura como debilidad que estas aerolíneas no pueden realizar viajes muy largos por el presupuesto de combustible ya planificado. Finalmente, el requisito 7, sincronización de calendario web, establece que los vuelos cargados desde la aplicación de escritorio en Java deben impactar automáticamente en la página web, de modo que los datos comerciales se actualicen en tiempo real en la plataforma del cliente sin tener que volver a ingresar la información, lo que evita la doble carga y los datos desactualizados.

Autenticación, roles y cuentas de usuario (requisitos 8 a 14)

Este bloque trata sobre quién puede entrar al sistema y qué puede hacer una vez adentro. El requisito 8, autenticación por roles, indica que el sistema valida las credenciales y separa los accesos en tres niveles: pasajeros, empleados y administradores. Cada tipo de usuario ve únicamente las pantallas y los datos que corresponden a su función, algo coherente con la tabla Usuario de la base de datos, que distingue a los usuarios según sus privilegios. El requisito 9, control de bloqueo por intentos, agrega que la ventana de inicio de sesión reduce un contador cada vez que se ingresa una credencial incorrecta y se cierra cuando el contador llega a cero, lo cual protege al sistema contra ataques de fuerza bruta, en los que alguien prueba contraseñas una tras otra.

El requisito 10, registro autónomo de empleados, establece que la ventana de creación de cuenta permite dar de alta usuarios con el rol de empleado por defecto, para que el personal operativo pueda generar su propio acceso inicial sin depender de un administrador. El requisito 11, validación de formato de usuario, admite en el identificador únicamente caracteres alfabéticos, tildes y guiones bajos, bloqueando números y espacios que podrían arruinar la estética de las pantallas o complicar las búsquedas. El requisito 12, garantizar

unicidad de credenciales, hace que el backend verifique en tiempo real que ni el nombre de usuario ni el correo electrónico existan previamente; esto cobra más sentido todavía después de haber agregado el campo email a la tabla Usuario el 18/09/26, ya que el correo pasó a ser un dato que identifica a cada cuenta.

Los requisitos 13 y 14 resuelven la recuperación de cuentas. El requisito 13, recuperación de cuenta por segundo factor, permite restablecer el acceso validando el correo electrónico y respondiendo una pregunta de seguridad, de forma que el usuario pueda desbloquear su cuenta por sí mismo sin esperar a un administrador. El requisito 14, validación robusta de nueva contraseña, obliga a que la nueva clave tenga como mínimo 8 caracteres y a que la confirmación coincida exactamente con lo escrito; es un estándar básico, pero necesario para que el mecanismo de recuperación no termine debilitando la seguridad del sistema.

Venta de pasajes (requisitos 15 a 17)

En la venta de pasajes me concentré en dos objetivos: reducir los errores del operador y asegurar que el dinero se calcule de manera exacta. El requisito 15, venta guiada de pasajes, obliga a que la interfaz gráfica haga seleccionar el vuelo y el pasajero mediante ventanas emergentes, de modo que no se pueda escribir a mano un código que no existe; el error de tipeo desaparece porque directamente no hay tipeo. El requisito 16, filtrado dinámico de servicios adicionales, hace que las casillas de verificación se generen a partir de las tarifas de los servicios activos, por lo que opciones como el equipaje extra se adaptan automáticamente a los precios vigentes en la base de datos sin modificar el código de la pantalla.

Este requisito quedó muy ligado a la decisión del 18/09/26 sobre el MVP, cuando eliminamos el cálculo del equipaje según el peso y resolvimos que el equipaje adicional se cobre únicamente como un servicio del catálogo: si el pasajero no contrata el servicio de equipaje extra, no puede llevar bultos adicionales. Esa decisión simplificó mucho la lógica, porque el

costo ya no depende de un algoritmo de pesos sino de una casilla que se tilda o no. El requisito 17, cálculo síncrono del costo total, establece que al emitir el pasaje el backend suma el precio base del boleto más el acumulado de los servicios elegidos, consolidando el valor final de la transacción de forma exacta y en el momento, y asegurando que el monto lo determine siempre un único lugar y no dependa de lo que muestre una pantalla.

Distribución y asignación de asientos (requisitos 18 a 20)

El requisito 18, mapeo interactivo del avión, define que la ventana de asientos debe dibujar una matriz visual simétrica de 180 lugares, diferenciados por color según estén libres u ocupados, para que el operador vea un plano gráfico real de quién está sentado en cada fila. La cifra de 180 no es casual: en el diseño inicial de la base de datos había 31 filas y 186 asientos por avión, y decidí reducirlo a 30 filas de seis asientos, de la A a la F, porque un número exactamente divisible hacía mucho más simple distribuir los botones y organizar la ventana en Java. En esa misma revisión cambié el formato de identificación de los asientos, de códigos como 1A a un formato uniforme de dos cifras como 01A, guardado en una columna código de tipo VARCHAR.

El requisito 19, validación de acceso al mapa de asientos, indica que el sistema debe bloquear la matriz si el pasajero no pagó el servicio adicional de selección de asiento, de manera que la pantalla solo sea interactiva cuando la regla comercial del boleto lo permite. Acá se refleja con claridad el modelo low cost: elegir asiento es un servicio con costo y la interfaz tiene que respetarlo. Para los pasajeros que optan por el check-in gratuito, el requisito 20, algoritmo de asignación aleatoria, hace que el backend busque automáticamente una plaza libre y se la asigne, repartiendo los lugares de forma matemática sin intervención humana.

Check-in y embarque (requisitos 21 y 22)

El requisito 21, emisión de pases de abordar, establece que el sistema registra el check-in vinculando al pasajero, el boleto, el vuelo y el pago, y de esa unión resulta la habilitación formal de embarque que reúne toda la información operativa anterior. Esto se corresponde con la tabla CheckIn de nuestro diagrama entidad-relación, que tiene claves foráneas hacia pasajero, boleto, vuelo y pago. El requisito 22, asignación de puertas de embarque, agrega que la interfaz de check-in debe permitir definir la puerta de salida, para que el operador pueda guiar físicamente al pasajero y quede registrado por dónde va a abordar el avión.

Pagos, auditoría y seguridad relacional (requisitos 23 a 25)

El requisito 23, control de estados de transacciones, indica que el módulo de pagos registra y actualiza el estado de cada operación bajo condiciones estrictas (Pagado, Pendiente, Cancelado, entre otras), lo que permite auditar el flujo financiero de cada pasaje vendido. El requisito 24, auditoría financiera en tiempo real, dispone que la interfaz muestre los costos desglosados en campos bloqueados y no editables: el operador puede ver los totales facturados pero tiene prohibido alterar los números de caja, una medida de transparencia que protege tanto a la empresa como al propio operador.

El requisito 25, seguridad relacional activa mediante disparadores, es el que cierra el bloque y trabaja del lado de la base de datos: mediante triggers se impide eliminar pasajeros o empleados que tengan registros históricos vigentes. Con esto se bloquean los fallos de consistencia que aparecerían si se borraran datos en cascada, por ejemplo, si desapareciera un pasajero que todavía tiene boletos y pagos asociados, dejando esos registros huérfanos. Quedó planificado junto con las vistas y los procedimientos almacenados para las próximas etapas del desarrollo de la base de datos.

Requisitos no funcionales

Los requisitos no funcionales no dicen qué hace el sistema, sino qué tan bien, qué tan seguro y qué tan ordenado está construido. Los 25 que definí los agrupé en seis temas: disponibilidad y rendimiento, integridad y seguridad de los datos, robustez del código Java, arquitectura y estilo de programación, infraestructura de red y organización del repositorio. A diferencia de los funcionales, aquí las pruebas consisten en revisar el código, los scripts SQL y la configuración de la red para comprobar que se respetan las normas establecidas.

Disponibilidad, concurrencia y rendimiento (requisitos 1, 2, 3 y 7)

El requisito 1, disponibilidad de la plataforma web, exige que el portal comercial garantice un uptime mínimo del 99,9 %, es decir que la página para los clientes esté operativa casi todo el tiempo, porque es la cara comercial de la aerolínea y una caída significa ventas perdidas. El requisito 2, tolerancia a la concurrencia, pide que la base de datos y la web soporten múltiples conexiones asincrónicas simultáneas, de forma que muchos usuarios puedan buscar y comprar boletos al mismo tiempo sin que el servidor colapse. El requisito 3, consumo de memoria RAM optimizado, establece que la aplicación Java de escritorio debe usar eficientemente los recursos locales, porque va a correr en las computadoras operativas de los aeropuertos y no puede volverlas lentas. Por último, el requisito 7, baja latencia en consultas secundarias, se refiere al submódulo de inteligencia competitiva por web scraping, que recolecta precios externos y debe operar con tiempos acotados para que esa recolección no demore los procesos internos de la empresa.

Integridad y seguridad de los datos (requisitos 4, 5, 8, 18 y 19)

El requisito 4, transacciones ACID, obliga al motor SQL a asegurar atomicidad, consistencia, aislamiento y durabilidad, de manera que los datos financieros y de asientos se graben por completo o no se graben, evitando registros corruptos. Esto es especialmente importante para impedir que dos pasajeros terminen con el mismo asiento. El requisito 8, integridad

referencial estricta, exige que las relaciones se validen mediante claves primarias y foráneas rigurosas, por lo que el motor impide cargar un boleto o un pago asociado a un pasajero o a un vuelo que no existen; es la base del esquema relacional que armé para las trece entidades del sistema.

El requisito 5, seguridad frente a inyecciones de código, obliga a que las consultas DAO utilicen sentencias preparadas (PreparedStatement), lo que bloquea los intentos de manipular la base de datos a través de los campos de texto. El requisito 18, restricciones de dominio rígidas, limita los campos de estado de pago y de check-in mediante tipos ENUM, de modo que la base rechaza cualquier texto que no coincida exactamente con las opciones permitidas. Y el requisito 19, seguridad activa en cascada, configura las bajas de usuarios en el histórico con la propiedad ON DELETE SET NULL: si se borra una cuenta de acceso, los registros contables y de auditoría no se eliminan ni se rompen, simplemente pierden la referencia a esa cuenta. Esto es posible porque en las tablas de pasajeros y empleados el campo usuario_id admite valores nulos.

Robustez del código Java (requisitos 6, 9 y 14)

El requisito 6, manejo local de excepciones, establece que el backend de Java capture las fallas mediante bloques try-catch estructurados, para que si ocurre un error con la base de datos el programa no se cierre abruptamente sino que avise al usuario. Esto se complementa con la decisión que tomé el 18/09/26 de aplicar la restricción NOT NULL en casi todas las columnas: si desde la aplicación se intenta guardar un dato incompleto, la base lanza una SQLException que se puede capturar y cuyo código de error permite identificar con precisión el origen del problema. El requisito 9, cierre automatizado de recursos, indica que las consultas de búsqueda deben usar bloques try-with-resources, para que las conexiones a la base de datos se liberen solas al terminar y se eviten las fugas de memoria. El requisito 14,

liberación eficiente de memoria volátil, pide que los flujos de navegación gráfica destruyan los marcos anteriores con el método dispose(), de manera que al cambiar de pantalla no se acumulen ventanas invisibles que sobrecarguen el equipo.

Arquitectura y estilo de programación (requisitos 10, 11, 12, 13, 15, 16 y 17)

El requisito 10, desacoplamiento de interfaces, exige que el portal web comercial esté completamente separado de la aplicación de escritorio, lo que permite modificar el diseño visual de la web sin tocar el código de gestión interno. El requisito 15, alta cohesión y bajo acoplamiento, indica que las clases comerciales resuelvan sus responsabilidades de manera nativa en sus constructores, para que un cambio en la lógica financiera no rompa ni afecte directamente a las pantallas. En la misma línea, el requisito 17, aislamiento de la persistencia, establece que el SQL embebido en Java debe reemplazarse por llamadas estructuradas a procedimientos almacenados, de modo que Java solo pide datos y delega las consultas pesadas al servidor MySQL. Para que esto sea fácil de mantener, el requisito 16, nomenclatura uniforme de parámetros SQL, obliga a que todos los parámetros de los procedimientos almacenados lleven el prefijo p_, así los desarrolladores distinguen enseguida las variables internas de las columnas reales de las tablas.

Los requisitos 11, 12 y 13 fijan el estilo. El requisito 11, uso de un Look and Feel unificado, exige que las pantallas Java inicialicen la librería externa FlatLaf, de modo que todas las ventanas del operador tengan una estética moderna, limpia y uniforme en escala de grises y azules; esto responde a la decisión del 16/09/26 de descartar el primer prototipo de ventanas y rediseñarlo con un aspecto más ordenado. El requisito 12, nomenclatura estandarizada en código, obliga a usar el estilo CamelCase en todo el desarrollo Java, lo cual se ve en las clases que ya armé en NetBeans bajo el paquete chorifly, como Aeropuerto, Avion, Asiento, Departamento, Persona, Empleado, Pasajero y Usuario. Y el requisito 13, estándar en base de

datos, unifica el uso de mayúsculas sostenidas para las palabras clave y los tipos de datos en los scripts SQL (VARCHAR, INT, DATE), para que las sentencias sigan una norma formal y se lean con prolijidad.

Infraestructura de red (requisitos 20, 21, 22 y 23)

Los cuatro requisitos de red salen del diseño que hicimos en Cisco Packet Tracer para las 20 subredes de la empresa. El requisito 20, tolerancia a fallos en el transporte de datos, establece que los sectores críticos, como operaciones y despacho, tengan topologías físicas en malla: si un router o un cable falla, la red sigue funcionando por caminos alternativos. El requisito 21, jerarquización de la infraestructura, indica que las subredes operativas se estructuren con switches multicapa y routers distribuidos, ordenando el tráfico del aeropuerto por sectores para evitar cuellos de botella. Esto es lo que cuantifiqué en los presupuestos de esta misma semana: 20 switches multicapa, 20 routers y 108 switches de acceso para los 20 sectores.

El requisito 22, redirección eficiente de paquetes broadcast, exige que los routers incorporen la directiva ip helper-address, que permite que las solicitudes dinámicas de DHCP crucen los diferentes sectores sin ser bloqueadas por los enrutadores, ya que de otro modo cada subred necesitaría su propio servidor DHCP. Por último, el requisito 23, escalabilidad de la infraestructura, pide que los routers Cisco admitan módulos de expansión físicos NIM-2T, de manera que la red pueda crecer e interconectar nuevas sedes sin cambiar los equipos principales. Esto se corresponde con la decisión de incorporar módulos seriales NIM-2T en los routers ISR 4331 para habilitar interfaces seriales físicas y el enrutamiento estático entre los segmentos.

Control de versiones y documentación (requisitos 24 y 25)

El requisito 24, organización semántica del repositorio, obliga a que el control de versiones en GitHub aplique de forma estricta la convención AA.B.C, para que el historial de archivos quede ordenado cronológicamente de forma exacta y resulte más sencillo auditarlo. El requisito 25, documentación técnica descentralizada, establece que cada rama (branch) de cada asignatura tenga su propio archivo README.md explicativo, de modo que cualquier auditor o profesor entienda el propósito y los archivos de cada materia de forma independiente. Este requisito lo fui cumpliendo desde que se crearon las branches y los README en Markdown, con títulos, listados de archivos y enlaces de navegación entre la rama principal y las demás.

Reflexión y conclusión del período

Al terminar de explicar los 50 requisitos comprobé que no son un listado aislado, sino un conjunto que se apoya en sí mismo: las reglas funcionales de los vuelos y los asientos dependen de la forma en que armé la base de datos, la seguridad del acceso se apoya en la validación del backend y en las sentencias preparadas, y la calidad del código depende de que todos usemos las mismas normas de nomenclatura. También noté que muchas de las decisiones que fuimos tomando a lo largo de las semanas anteriores, como el límite de un vuelo programado por avión, los 180 asientos, el cobro del equipaje como servicio o la incorporación de los módulos NIM-2T, tienen su justificación directa en alguno de estos requisitos, lo cual me da seguridad de que el diseño es coherente.

La principal dificultad de este período fue redactar los requisitos de modo que fueran verificables y no quedaran ambiguos, sobre todo los no funcionales, porque enunciados como «consumo de memoria optimizado» o «alta cohesión» son más difíciles de medir que una regla como «bloquear tramos mayores a 5 horas». Para resolverlo, acompañé cada enunciado con la explicación de lo que significa en la práctica. Considero que esta tarea me permitió

entender el sistema en profundidad, y me sirve de base tanto para las pruebas de QA y los test unitarios como para la documentación final, la exposición del proyecto y las próximas etapas del desarrollo, que son los procedimientos almacenados, los triggers, las vistas, la conexión de Java con la base de datos y las interfaces definitivas.